IIT
INSTITUT INTERNATIONAL TECHNOLOGIE
Université Nord Américaine privée

dqs ISO 21001: 2018 ASIIN Euro-Inf Bachelor awarded by EQANIE

TRAVAUX PRATIQUES 2

DM & MACHINE LEARNING



2ème année Génie Informatique SDIA/GLID

Semestre 1 ♣

M. Taoufik BEN ABDALLAH

taoufik.benabdallah@iit.ens.tn

M. Radhouane Guermazi

rguermazi@gmail.com

2026-2027

Correction TP2

```
import warnings
warnings.filterwarnings("ignore",category=FutureWarning)
```

1/ Charger le jeu de données **titanic.csv** dans une variable nommée **df_titanic**. Afficher le nombre total d'observations ainsi que les **5** premières lignes de **df_titanic**

```
df_titanic=pd.read_csv('/…/titanic.csv')

print(len(df_titanic)) #891

df_titanic.head()
```

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Survived
0	male	22.0	1	0	A/5 21171	7.250	NaN	S	0
1	female	38.0	1	0	PC 17599	NaN	C85	C	1
2	female	26.0	0	0	STON/02. 3101282	7.925	NaN	S	1
3	female	35.0	1	0	113803	53.100	C123	S	1
4	male	35.0	0	0	373450	NaN	NaN	S	0

2/ Afficher le type de chaque attribut de **df_titanic** en indiquant le **nombre d'observations non-null pour chaque attribut**

```
df_titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 9 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Sex       891 non-null    object
 1   Age       714 non-null    float64
 2   SibSp     891 non-null    int64
 3   Parch     891 non-null    int64
 4   Ticket    891 non-null    object
 5   Fare      888 non-null    float64
 6   Cabin     204 non-null    object
 7   Embarked  889 non-null    object
 8   Survived  891 non-null    int64
dtypes: float64(2), int64(3), object(4)
```

Correction TP2

3/ Afficher un résumé statistique de toutes les colonnes de **df_titanic**

```
df_titanic.describe(include='all')
```

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Survived
count	891	714.000000	891.000000	891.000000	891	888.000000	204	889	891.000000
unique	2	NaN	NaN	NaN	681	NaN	147	3	NaN
top	male	NaN	NaN	NaN	347082	NaN	B96 B98	S	NaN
freq	577	NaN	NaN	NaN	7	NaN	4	644	NaN
mean	NaN	29.699118	0.523008	0.381594	NaN	32.188447	NaN	NaN	0.383838
std	NaN	14.526497	1.102743	0.806057	NaN	49.753475	NaN	NaN	0.486592
min	NaN	0.420000	0.000000	0.000000	NaN	0.000000	NaN	NaN	0.000000
25%	NaN	20.125000	0.000000	0.000000	NaN	7.895800	NaN	NaN	0.000000
50%	NaN	28.000000	0.000000	0.000000	NaN	14.454200	NaN	NaN	0.000000
75%	NaN	38.000000	1.000000	0.000000	NaN	31.000000	NaN	NaN	1.000000
max	NaN	80.000000	8.000000	6.000000	NaN	512.329200	NaN	NaN	1.000000

4/ Déterminer le **nombre total de valeurs manquantes** (**NaN**) ainsi que celui **pour chaque attribut**

```
print("Nombre total de valeurs manquantes=",
          df_titanic.isnull().sum().sum())

print("\nNombre de valeurs manquantes pour chaque attribut= ")

df_titanic.isnull().sum()
```

```
Nombre total de valeurs manquantes= 869

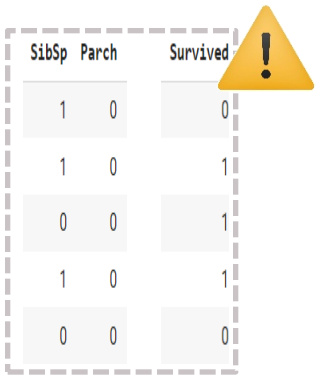
Nombre de valeurs manquantes pour chaque attribut=
Sex          0
Age        177
SibSp        0
Parch        0
Ticket       0
Fare         3
Cabin      687
Embarked     2
Survived     0
dtype: int64
```

Correction TP2

5/ Créer deux listes, **l_discret** et **l_num**, qui contiennent respectivement les noms des colonnes avec des **valeurs catégorielles** et **numériques** dans **df_titanic**

```
l_num=list(df_titanic.select_dtypes(include="number"))
#['Age', 'SibSp', 'Parch', 'Fare', 'Survived']

l_discret=list(df_titanic.select_dtypes(include="object"))
#['Embarked', 'Sex', 'Ticket', 'Cabin']
```



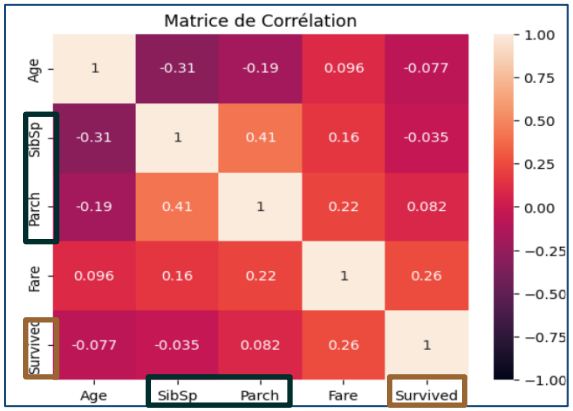
6/ Présenter la matrice de corrélation de Pearson entre les différents attributs numériques de **df_titanic**

```
df_corr=df_titanic.corr(numeric_only=True)

sns.heatmap(df_corr, annot=True, vmin=-1, vmax=1)

plt.title('Matrice de Corrélation')

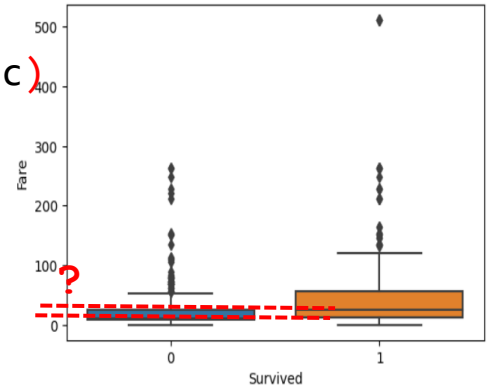
plt.show()
```



Correction TP2

7/ Représenter les boxplots de l'attribut **"Fare"** de **df_titanic**, en fonction des valeurs de l'attribut numérique qui présente la corrélation la plus élevée. Ensuite, remplir les **valeurs manquantes** de l'attribut **"Fare"** avec les valeurs appropriées

```
sns.boxplot(x='Survived',y='Fare',data=df_titanic)
plt.show()
```






```
a=df_titanic[df_titanic["Survived"]==0]['Fare'].median()
b=df_titanic[df_titanic["Survived"]==1]['Fare'].median()
```

```
df_titanic['Fare']= df_titanic['Fare'].fillna(df_titanic['Survived'].apply(lambda x: a if x == 0 else b), inplace=True)
```

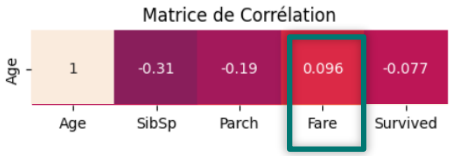
```
med_classe = df_titanic.groupby('Survived')['Fare'].median()
df_titanic['Fare']= df_titanic['Fare'].fillna(df_titanic[' Survived '].apply(lambda x: med_classe[x]),inplace=True)
```

Correction TP2

8/ Trier **df_titanic** **en ordre décroissant** en fonction de l'attribut ayant une **forte corrélation positive** avec **"Age"**. Ensuite, remplir les valeurs manquantes de l'attribut **"Age"** de **df_titanic** avec les valeurs de la cellule précédente dans la même colonne

```
df_titanic.sort_values(by="Fare",ascending=False,inplace=True, ignore_index=True)
```

```
df_titanic['Age']=df_titanic['Age'].ffill()
```



9/ Remplir les valeurs manquantes de l'attribut **"Cabin"** de **df_titanic** avec l'élément le plus fréquent

```
imputer = SimpleImputer(missing_values=np.nan, strategy='most_frequent')
```

```
df_titanic[['Cabin']]=imputer.fit_transform(df_titanic.loc[:,['Cabin']])
```

Correction TP2

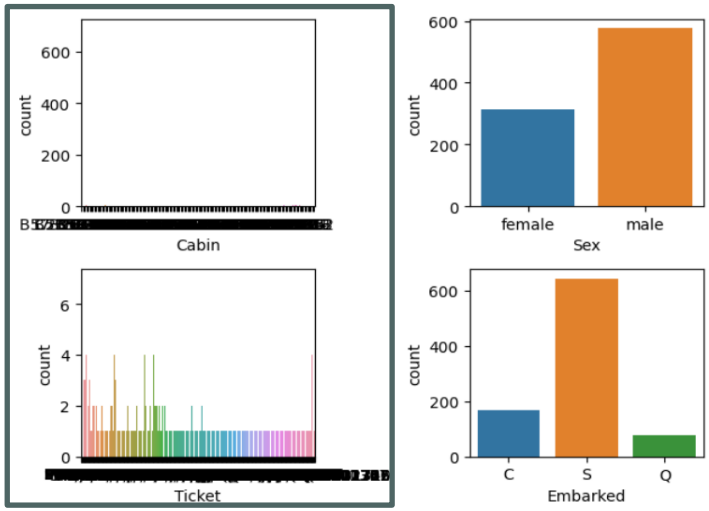
10/Supprimer **les observations ayant des valeurs manquantes** dans l'attribut **"Embarked"** de **df_titanic**

```
df_titanic.dropna(subset=['Embarked'],inplace=True)
```

```
df_titanic.reset_index(drop=True, inplace=True)
```

11/ Créer une figure avec **4** axes où chaque ligne comporte **2** axes : chacun de ces axes affiche l'**histogramme de la distribution d'un attribut catégoriel** à partir de **df_titanic**. Ensuite, **supprimer** l'attribut ou les **attributs discret(s) que vous jugez inutiles** à partir de **df_titanic**

```
for i in range(len(l_discret)):
  plt.subplot(2,2, i+1)
  sns.countplot(x=l_discret[i], data=df_titanic)
plt.tight_layout()
plt.show()
```




```
df_titanic.drop(labels=["Cabin","Ticket"], axis=1, inplace=True)
```

Correction TP2

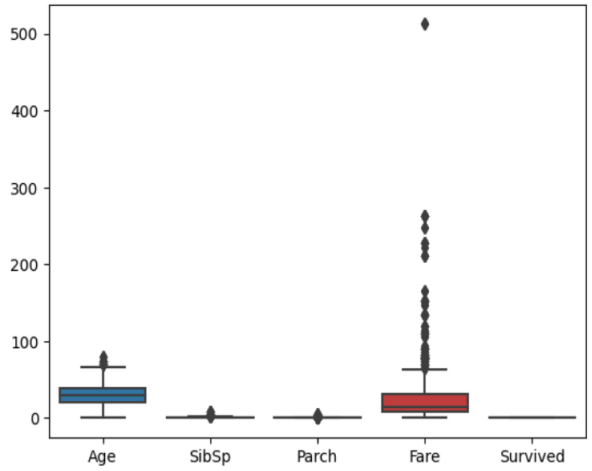
12/ Vérifier que **df_titanic** ne comporte aucune valeur manquante et **traiter les données aberrantes de tous les attributs numériques** de **df_titanic** **1/2**

```
df_titanic.isnull().sum()
```

```
Sex          0
Age          0
SibSp        0
Parch        0
Fare         0
Embarked     0
Survived     0
dtype: int64
```

```
sns.boxplot(data=df_titanic[l_num])

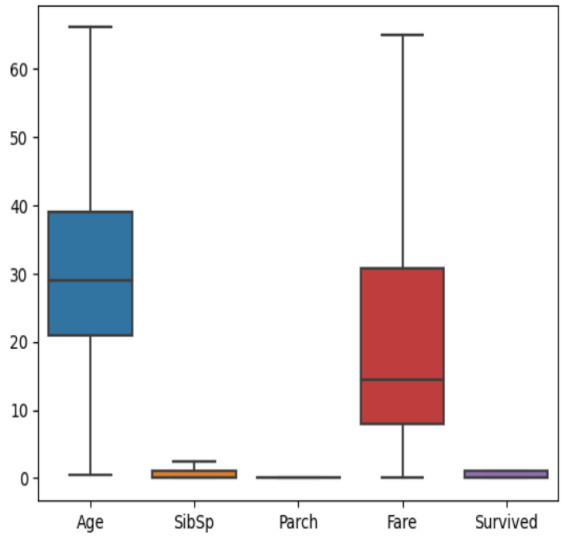
plt.show()
```



Correction TP2

12/ Vérifier que **df_titanic** ne comporte aucune valeur manquante et **traiter les données aberrantes de tous les attributs numériques** de **df_titanic** **2/2**

```
def adjust_outliers(df, col):
  Q1,Q3=np.percentile(df[col], [25,75])
  IQR=Q3-Q1
  upper_limit=Q3+1.5*IQR
  lower_limit=Q1-1.5*IQR
  df[col]=np.where(df[col]>upper_limit, upper_limit, np.where(df[col]<lower_limit,
            lower_limit,df[col]))
```




```
adjust_outliers(df_titanic, "Age")
adjust_outliers(df_titanic, "SibSp")
adjust_outliers(df_titanic, "Parch")
adjust_outliers(df_titanic, "Fare")
```

Correction TP2

13/ Encoder les valeurs des attributs **"Sex"** (**female**, **male**) et **"Embarked"** (**C**, **Q** et **S**) de **df_titanic** en valeurs numériques ordinales

```
enc = OrdinalEncoder()

df_titanic[['Sex',"Embarked"]]=enc.fit_transform(

        df_titanic.loc[:,["Sex", "Embarked"]])
```

	Sex	Embarked
0	female	C
1	male	C
2	male	C
3	female	S
4	male	S
...	...	...
884	male	S
885	male	S
886	male	S
887	male	S
888	male	S

	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
0	0.0	2.0	0.0	0.0	64.8958	0.0	1
1	1.0	2.0	0.0	0.0	64.8958	0.0	1
2	1.0	2.0	0.0	0.0	64.8958	0.0	1
3	0.0	1.0	2.5	0.0	64.8958	2.0	1
4	1.0	1.0	2.5	0.0	64.8958	2.0	0
...	...	...	...	...	...	...	...
884	1.0	2.0	0.0	0.0	0.0000	2.0	0
885	1.0	2.0	0.0	0.0	0.0000	2.0	0
886	1.0	2.0	0.0	0.0	0.0000	2.0	0
887	1.0	2.0	0.0	0.0	0.0000	2.0	0
888	1.0	2.0	0.0	0.0	0.0000	2.0	0

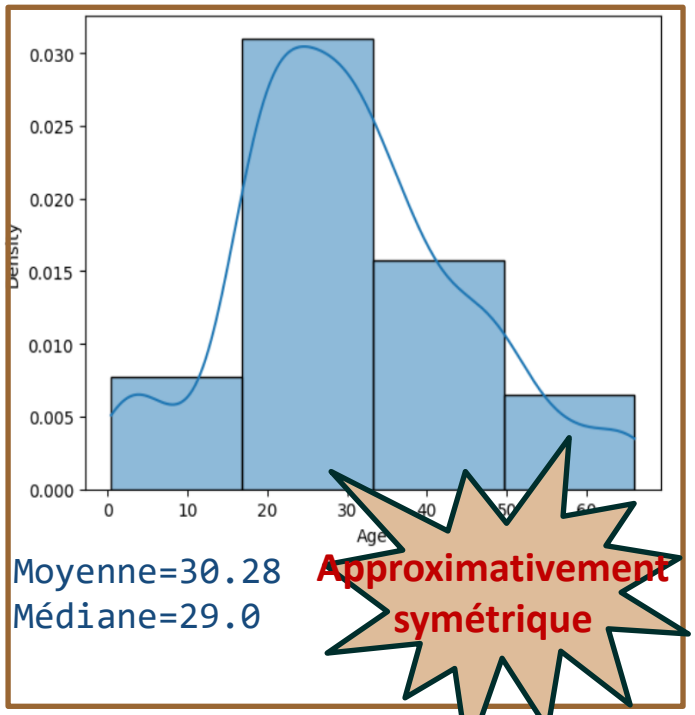
Correction TP2

14/ Afficher un **histogramme** illustrant la **distribution des âges** de **df_titanic** selon **4** intervalles, avec une **courbe de densité gaussienne** intégrée dans la même figure. Ensuite, discrétiser les valeurs en utilisant un encodage ordinal, en choisissant la technique appropriée

```
sns.histplot(df_titanic['Age'], kde=True, stat='density', bins=4)
```

```
enc=KBinsDiscretizer(n_bins=4,  encode='ordinal',
          strategy='quantile', subsample=len(df_titanic))
```

```
df_titanic[['Age']]= enc.fit_transform(df_titanic.loc[:,['Age']])
```



Correction TP2

15/ Sélectionner **les descripteurs les plus importants** à partir de **df_titanic** en appliquant la technique de la `variance`, avec un seuil de `0.1`

```
s=VarianceThreshold(threshold=0.1)
arr= s.fit_transform(df_titanic)
# Obtenir les indices des colonnes non-supprimées
ind_col= s.get_support(indices=True) #[0 1 2 4 5 6]
df_titanic=pd.DataFrame(arr,columns=list(df_titanic.iloc[:,ind_col]))
```

	Sex	Age	SibSp	Fare	Embarked	Survived
0	0.0	2.0	0.0	64.8958	0.0	1.0
1	1.0	2.0	0.0	64.8958	0.0	1.0
2	1.0	2.0	0.0	64.8958	0.0	1.0
3	0.0	1.0	2.5	64.8958	2.0	1.0
4	1.0	1.0	2.5	64.8958	2.0	0.0
...	...	...	...	...	...	...
884	1.0	2.0	0.0	0.0000	2.0	0.0
885	1.0	2.0	0.0	0.0000	2.0	0.0
886	1.0	2.0	0.0	0.0000	2.0	0.0
887	1.0	2.0	0.0	0.0000	2.0	0.0
888	1.0	2.0	0.0	0.0000	2.0	0.0

889 rows × 6 columns

Exporter en **CSV** le DataFrame résultat de **df_titanic** sous le nom ***titanic_preprocess.csv***

```
df_titanic.to_csv("/…/titanic_preprocess.csv", index=False)
```

TP2 Autre question

Q/ Supprimer les observations ayant des valeurs NaN dans tous les descripteurs *qui ont au moins une valeur manquante*

```
S=df_titanic.isnull().sum()
```

```
df_titanic.dropna(subset=list(s[s!=0].index), how="all", axis=0, inplace=True)
```